打造稳定的 AI 开发工作环境

AI Coding Agent 实战工作坊 — 中阶场 (Claude)

目标： 了解 Prompt、Context、Harness 如何配合，让 AI 在团队里更稳定地工作

John Wu | Apr 2026

今天的旅程

时段	做什么
观念讲解	用餐厅比喻看 Prompt / Context / Harness
Live Demo	看同一个需求如何一步步变稳
练习时间	套到你自己的工作场景
回顾 Q&A	重点整理 + 课后练习说明

> 重点不是多学几个名词，而是知道怎么把 AI 放进可重复的工作流程里

Part 1: 观念讲解

用「开餐厅」来比喻

你说的话	餐厅比喻	对应概念
「我想吃辣的」	先做一版试吃再调整	Vibe Coding
「宫保鸡丁，不要花生，少油」	讲越具体，出餐越准	Prompts
「这是川菜馆，大火快炒」	厨房规矩写好	Project Instructions
食谱 + 检查点 + 工具清单	标准作业流程	Skill
接上供应商、仓库、外送系统	对外接口	MCP

今天要升级什么？

角度	你在做什么
单次协作	把需求讲清楚，让 AI 帮你完成一次任务
可重复流程	设计规则、流程、工具，让 AI 能稳定完成很多次任务

> 你不只是想做出一道菜，而是开始设计整间厨房怎么稳定运作
> 同样一道菜，有人每次都靠运气，有人靠制度稳定出餐

今天的工具视角

工具	今天的定位
Claude Code	主要示范工具，展示怎么设计 AI 工作环境
Codex	偶尔对照，帮你确认概念是跨工具通用的

重点不是比工具，而是看原理怎么搬

这些概念怎么搬到你熟悉的工具里

> 工具名称会变，但 Prompt / Context / Harness 这套思路很稳

AI 工作环境的三大支柱

支柱	餐厅比喻	白话
Prompt	你怎么点餐	你给 AI 的指令、限制、输出要求
Context	菜单、厨房规矩、食材信息	AI 获取到的项目知识与外部信息
Harness	厨房制度、权限、安全检查	AI 可以怎么做、做到哪里、何时该停下来

> 中阶不是学更多术语，而是知道哪一层没补好

为什么同一句需求，结果还是会飘？

常见原因通常不只一个

1. **Prompt 不够清楚**：任务、限制、输出格式没有讲明
2. **Context 不够完整**：AI 不知道 repo 规范、现有 code、外部数据
3. **Harness 不够稳**：没有检查点、权限边界或 workflow 约束

所以你要学会的不是「写超长 prompt」

而是判断：这次要补的是哪一层

> 问题不一定出在你不会问，也可能出在厨房根本不知道规矩

Prompt — 你怎么点餐

不是每件事都要用同样松紧度

任务类型	点餐方式	建议松紧度
数据库迁移	指定做法、限制风险	**严格**
API 实现	讲清规格，保留部分空间	**中等**
Code Review	给目标与观察角度	**中等到宽松**
写文档	讲清受众与格式	**宽松**

> 越危险的操作，prompt 越要明确；越探索的任务，可以留多一点空间

Context — 厨房知道什么

如果 Prompt 是点餐，那 Context 就是：

- 这是一家什么餐厅
- 厨房有什么规矩
- 现在有哪些食材
- 有没有外部系统可以查数据

```
常见 context = 对话内容 + 相关文件 + 项目指引文件 + 需要时取得的外部信息
⚠️ context 不只影响质量，也影响成本与效率——塞太多不相关的东西，判断会变差、资源也会浪费
```

> AI 不只是听你当下说什么，还会受它拿得到的信息影响

Context — 项目指引文件是厨房规矩

概念	Claude Code	Codex	Gemini
项目指引文件	`CLAUDE.md`	`AGENTS.md`	`GEMINI.md`
作用	告诉 AI 项目身份、规范、工作方式	同类型概念	同类型概念
今天定位	主角	对照示例	对照示例

你是在跟 AI 说：「这是我们家的厨房，我们平常就是这样做事」

> 写一次，之后每次对话都比较容易站在同一套规则上

Context — 项目指引文件通常写什么？

```
# Project Name

## Description
(...)

## Tech Stack
React + Vite, Express, MongoDB

## Coding Conventions
- 组件用 functional component
- API 回传统一用 { data, error }

## Workflow
- 改 UI 前先确认 RWD breakpoint
- API 改完要跑 integration test
```

不是在列百科全书，而是在定义这间厨房的做事方式

> 写太少，AI 会乱猜
> 写太多，真正重要的规则被淹没

⚠️ ETH Zurich 2026：

AI 自己生的指引档 → 成功率反而**下降**

人写但塞太多 → 几乎没帮助

> 只写最小必要规则，效果最好

Context — 项目指引文件怎么写才有用？

要写	不要写
规则（约束怎么工作、什么不能做）	介绍（项目是什么、目录结构）
团队/仓库独有的限制	模型本来就会的常识
长期有效的流程	临时任务（应写成 Skill 或 Hook）
反面示例「不要这样做」	正面套话「保持优雅」

> 每一行都占全局上下文
> 不写就会出错的才写进去，同样错误出现两次再更新

Context — Prompt 其实是组装出来的

你写的不只是一份 CLAUDE.md，而是可组合的模块

```
CLAUDE.md → role → policy → stack → skills → templates
```

每次都载入（核心）　　按需选用

- **核心模块**：身份、规则 — 每次对话都带着
- **Skills**：怎么做 — 任务有固定 SOP 时载入
- **Templates**：结构化的信息格式 — 任务背景、报告、提案都是模板

Harness — 厨房怎么被管住

机制	餐厅比喻	说明
权限 / approval	哪些事要店长批准	高风险操作先确认
Hooks / 自动检查	出餐前先过品管	在特定时机自动跑检查或流程
Workflow 约束	厨房标准流程	先看 code、再改、最后验证

```
// .claude/settings.json — hook 示例
// 白话：「每次 AI 改完文件，自动跑一次代码检查」
{ "hooks": { "PostToolUse": [{
    "matcher": "Edit",
    "hooks": [{ "type": "command", "command": "pnpm lint --fix" }]
}]}}
```

> Harness 像安全带 — 不是绑手绑脚，而是让你敢放手让 AI 跑

Harness 的两个面向

面向	做什么	餐厅比喻
自动护栏	hooks、权限、检查点，防止出错	出餐前品管自动检查
反复调校	根据结果回头修正 Prompts & Skills	每次出餐后检讨，调整食谱和流程

两者搭配才完整

- 只有护栏，没有调校 → 规则僵化，遇到新状况就卡住
- 只有调校，没有护栏 → 每次都靠人盯，效率上不去

> 执行期护栏与事后调校要分开看，但两者都会影响整体 workflow 的稳定度

Skills / MCP / Plugins，分别在做什么？

比喻	概念	白话
点餐方式	Prompt	你怎么下指令
厨房规矩	CLAUDE.md / AGENTS.md	项目背景与做事方式
标准作业流程	Skills	让 AI 按步骤完成一种任务
对外接口	MCP	让 AI 连到外部工具或数据
整套设备包	Plugins	把常用能力打包成可重用组合

> 你是在设计一间会运作的厨房，不是在多买几个厨具

Skills — 把做事方法变成 SOP

Skill 可以先把常见任务的流程写好，例如：

- 先收敛需求
- 再拆计划
- 接着实现
- 最后验证与收尾

```
模糊需求 -> brainstorming -> plan -> implementation -> verification
```

> Skill 不只是 prompt 模板，而是可重复套用的工作流程

MCP — 让厨房自己去拿信息

MCP 可以让 AI 在需要时自己去查数据或做操作，例如：

- 读 issue 或 ticket
- 查数据库结果
- 看团队讨论脉络
- 读文件或文档

关键不是「接了多少系统」

而是：**AI 能不能在对的时候，自己拿到它需要的信息**

> MCP 是对外接口，不是外部系统本身

一句话收敛：中阶到底在学什么？

Prompt

把需求讲清楚

Context

让 AI 知道这间厨房的规矩与信息来源

Harness

让 AI 在对的边界与流程下工作

> 中阶不是多装工具，而是开始设计 **AI 的工作环境**

Part 2: Live Demo

从 0 到 1，让 AI 按流程做出产品雏形

```
做一个 RWD 的任务管理系统
需要有任务列表、状态筛选、新增任务表单
桌机与手机都要可用
```

Demo 怎么对应三大支柱？

- **Prompt / Context**：先定角色与规则
- **Context → Spec**：用 brainstorming 把需求收敛
- **Harness**：用 plan、反馈循环推进与修正

Demo 1 — 先建立 Role / Skill 系统

在项目中建立三个 Role Prompts

- **Frontend**：专注 UI 结构、RWD、互动与组件拆分
- **Backend**：专注 API、数据结构、验证与错误处理
- **QA**：专注测试案例、边界条件与风险提醒

每个角色再配一个 Skill

- **Frontend skill**：先定页面结构，再定 breakpoint，再拆组件
- **Backend skill**：先定 schema，再定 API，再补 validation
- **QA skill**：先写 test plan，再补 edge cases 与 E2E checklist

Demo 2 — 建立约束条件

用 `/init` 产生 CLAUDE.md，再把规则写进去

```
/init 这个项目每次执行任务时，只带一个角色与对应 Skill，Skill 最多 3 个
```

情境	没约束	有约束
角色混用	前后端混扮，风格不一致	一次一个角色，职责清楚
Skill 爆量	太多 SOP 互相冲突	最多 3 个，聚焦当前任务
规则遗失	每次对话都要重新交代	写进 CLAUDE.md，自动生效

Demo 3 — Brainstorm 把需求收斂成 Design Spec

安装 Superpowers

从一句模糊需求开始

```
/superpowers:brainstorm 我想做一个 RWD 的任务管理系统
```

- 逐步澄清成可落地规格
- 决定架构方案、技术栈

> 关键不是直接写 code，而是把需求先收斂成可以分工与验收的 spec

Demo 4 — 从 Plan 到逐步实现

先把 spec 变成可执行计划

- 把 spec 拆出 **N 个 tasks**
- 依序让 AI 实现每个 task
- 最后用 build / test 当验收标准

> AI 依照 plan 一步步推进，每完成一步就验证再继续

执行顺序

1. 后端 schema + API
2. 前端组件 + 页面
3. QA 补测试案例与验收

Demo 5 — 根据反馈即时修正

启动环境验证

- 请 AI 修改完自动启动 backend 与 frontend
- 用浏览器实际操作，检查功能是否可用

问题	该补什么	修正方式
手机版任务卡片排版跑掉	Context（规范）	调整 flex / media query
新增任务后 API 回 404	Prompt（路由细节）	检查路由与 controller
筛选「已完成」仍显示全部	Harness（测试检查点）	确认 query 与 filter

> 成品不会一次到位，关键是判断该补哪一层，再反馈给 AI

Demo 6 — 用 OpenSpec 提出需求变更

1. 安装 OpenSpec（`/opsx` 是它产生的 slash command 前缀）
2. 项目中启用 `openspec init`
3. `/opsx:explore`：提出需求
4. `/opsx:propose`：建立需求文档及分工规划
5. `/opsx:apply`：依照 spec 实现
6. `/opsx:archive`：验收完成后，把需求归档

> OpenSpec 不是文档仓库，而是需求、设计、实现、验收的变更账本

Demo 回顾

重点	要让观众看到什么
Role & Skill 系统	先定角色，再定 SOP，最后用规则把 AI 约束住
Superpowers	模糊需求可以被逐步收敛成可执行规格
Plan 驱动实现	把 spec 拆成 task，依序实现、逐步验收
即时反馈修正	判断该补 Prompt / Context / Harness 哪一层
OpenSpec 工作流	`explore → propose → apply → archive` 让每次变更都有脉络
Feedback Loop	用计划、验证与反馈循环，让流程越跑越稳

除了 Skill，还有一个好用的层：Templates

模板	用途
`templates/task-context.md`	单次任务背景（最轻量）
`templates/change-context.md`	单一变更（含范围、影响）
`templates/project-context.md`	项目全貌（最完整）
`templates/code-review.md`	Code Review 报告
`templates/design-proposal.md`	设计提案

> 每个模板都是「一种结构化的信息格式」

Part 3: 练习时间 (20 min)

把今天的方法套到你的工作环境

> 练习 1 为必做，练习 2 选做，自由实现为课后延伸

练习 1 — 设计你团队的厨房规矩

你的任务

替你的 repo（或今天的 demo 练习项目）草拟一份 `CLAUDE.md`

至少回答这四题

1. 这是什么项目？用什么技术？
2. AI 改 code 前，应先知道哪些规范？
3. AI 做完后，至少要验证什么？
4. 有哪些事不能乱做？

> 先写出 60 分版本，比追求一次写到 100 分更实际

练习 2 — 替常见任务补上 Harness

想一个你们团队常做的任务，然后定义权限与检查点

要回答的问题	示例
哪些操作可以直接做？	读文件、跑 lint
哪些操作应该先问？	删除文件、改 DB schema
做完之后怎么验证？	跑 test suite、检查 API response

> 很多翻车不是因为 AI 不够聪明，而是因为没有安全带

自由实现（课后延伸）

选一个最接近你工作的题目

> A) 为团队 repo 草拟一份 `CLAUDE.md`
> B) 把一个模糊需求拆成 Prompt / Context / Harness
> C) 找一个常见任务，设计它的 Skill（从触发条件到验收标准）

提醒

做出来的内容最好是你回去就能直接用的

> 不求做很多，先做出一个能落地的版本

Part 4: 回顾

实务提醒：成本与 context 意识

前面提过，context 不只影响质量，也影响成本

这些情况通常都比较耗资源：

行为	常见影响	建议
超长对话	context 越来越重	适时开新对话，保持主题干净
一次塞太多不相关信息	判断质量下降	只给真正相关的内容
反复试错但不收敛	花时间也花额度	退一步先补 context 或规则
不必要的多工具串接	复杂度上升	先解决核心问题，再加工具

> 省资源的方法通常不是少问，而是让每次提问更有结构

关键 Takeaway

1. Prompt + Context + Harness 三者缺一不可

好的结果，通常不是靠单一神 prompt，而是三层一起补

2. `CLAUDE.md` / `AGENTS.md` 是团队共享的厨房规矩

写一次，之后每次合作都不用重新对齐规矩

3. Skills + MCP 让 AI 从聊天工具变成工作环境的一部分

不只会回答，还能按流程做事、在需要时自己拿信息

> 中阶的核心不是「更会用 AI」，而是「更会设计 AI 怎么工作」

课后练习

练习	怎么做
写一版项目指引文件	先列技术栈、规范、验证流程、禁区
做一次四轮补强实验	同一任务依序只补 Prompt → Context → Harness → Skill (比较结果差异)
找一个常见流程做 SOP	先用文字写出步骤，再考虑要不要做成 Skill
结果不稳时怎么判断	回头看是 Prompt / Context / Harness 哪一层没补好

回去先做一件事

> **在你的 repo 里，补上最基本的 AI 工作规矩**

下一站：进阶场预告

今天你学的是——把 AI 放进稳定的工作环境

进阶场会学的是——把多个 Agent 组成可持续运作的流程

中阶（今天）	进阶（下一场）
餐厅：一间厨房怎么运作	工厂：多条产线怎么协调
单一 Agent + 规则	多 Agent 架构 + 自订 Skill
Prompt / Context / Harness	Agentic Workflow 设计

> 今天打好的基础（角色、规则、检查点），进阶场会全部用上

Q&A